Análise de Desempenho das Cláusulas `collapse` e `schedule`

Simulação da Equação de Difusão 2D

Werbert Arles de Souza Barradas

Universidade Federal do Rio Grande do Norte (UFRN)
Disciplina de Programação Paralela - DCA3703

28 de setembro de 2025

Agenda

1. Introdução e Estratégias
2. Implementações
3. Validação do Código Serial
4. Resultados e Análise
5. Conclusão

Contexto e Estratégias Avaliadas

Problema

O custo computacional de simulações científicas está concentrado em **laços aninhados**. O objetivo é analisar o impacto de diferentes estratégias de paralelização de laços em OpenMP para um simulador de difusão 2D, com 6000 repetições(NT=6000)e grade de 512x512(NX=NY).

Versões Analisadas

1. **Serial:** Implementação sequencial para baseline.
2. **Paralelismo Ingênuo:** Múltiplas regiões paralelas (alto overhead).
3. **Otimização Parcial:** Região paralela única, mas **sem** `collapse`.
4. **Otimização Completa:** Região única **com** `collapse(2)` e teste das políticas de `schedule`.

1. Implementação Serial (Baseline)

Propósito

Estabelecer o tempo de execução base (T_{serial}). Todos os laços são sequenciais.

```
// Loop de tempo PRINCIPAL - SERIAL
for (int t = 0; t < NT; t++) {
    // La os espaciais - SERIAIS
    for (int i = 1; i < NX-1; i++) {
        for (int j = 1; j < NY-1; j++) {
            // ... c lculo do Laplaciano ...
        }
    }
}
```

2. Paralelização Ingênua

Estratégia e Gargalo

Aplicar `#pragma omp parallel for` em cada laço de trabalho. **Gargalo:** Alto overhead de criar e destruir threads (fork/join) 3x a cada passo de tempo.

```
// Loop de tempo PRINCIPAL - SERIAL
for (int t = 0; t < NT; t++) {
    // GERA OVERHEAD DE FORK/JOIN AQUI
    #pragma omp parallel for
    for (int i = 1; i < NX-1; i++) { /*...*/ }

    // E AQUI...
    #pragma omp parallel for
    for (int i = 0; i < NX; i++) { /*...*/ }

    // E AQUI NOVAMENTE...
    #pragma omp parallel for
    for (int j = 0; j < NY; j++) { /*...*/ }
}
```

3. Otimização Parcial (sem collapse)

Estratégia e Gargalo

Criar uma **única região paralela** para eliminar o overhead de fork/join. **Gargalo Remanescente:** Sem `collapse`, o OpenMP só distribui as linhas (laço `i`), limitando o balanceamento de carga.

```
// Loop de tempo PRINCIPAL - PARALELO
#pragma omp parallel
{
    for (int t = 0; t < NT; t++) {
        // UMA  NICA  CRIA  O DE THREADS AQUI

            // AINDA SEM COLLAPSE: paraleliza apenas o la o 'i'
            #pragma omp for schedule(static)
            for (int i = 1; i < NX-1; i++) {
                for (int j = 1; j < NY-1; j++) { /*...*/ }
            }
            #pragma omp for schedule(static)
            for (int i = 0; i < NX; i++) { /*...*/ }
    }
} // FIM DA REGI O PARALELA
```

4. Paralelização Otimizada (com collapse + schedule)

Estratégia Final

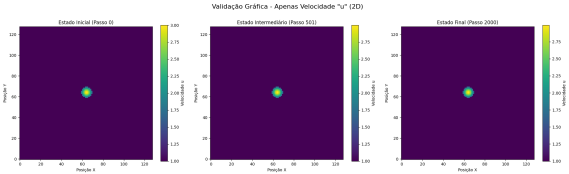
Manter a região paralela única e adicionar `collapse(2)` para tratar a grade 2D como um único bloco de trabalho, maximizando o potencial de balanceamento de carga.

```
// Loop de tempo PRINCIPAL - PARALELO
#pragma omp parallel
{
    for (int t = 0; t < NT; t++) {
            // MELHOR BALANCEAMENTO DE CARGA
            #pragma omp for collapse(2) schedule(static)
            for (int i = 1; i < NX-1; i++) {
                for (int j = 1; j < NY-1; j++) { /*...*/ }
            }

            #pragma omp for schedule(static)
            for (int i = 0; i < NX; i++) { /*...*/ }

    }
}// FIM DA REGI O PARALELA
```

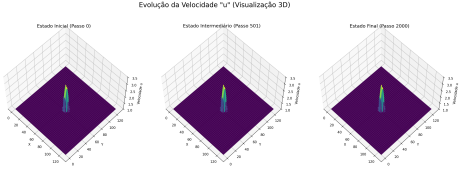
Validação Gráfica da Simulação

Análise de Coerência

Antes de analisar o desempenho, validamos se a simulação serial produz resultados fisicamente estáveis. Os gráficos mostram a evolução do campo de velocidade "u" em três instantes (passos 0, 501 e 2000), confirmando a estabilidade do código que servirá de *baseline*.



Visualização 2D



Visualização 3D

Resultado

A simulação se mostra estável, sem divergir ou apresentar artefatos, validando o código base.

Resultados Quantitativos (Tempos de Execução)

Estratégia	N. Threads	Tempo Médio (s)
Serial	**1**	**4.7540**
Paralelo Ingênua	16	3.0866
Otimizado (schedule + static)	16	2.8732
Otimizado (schedule + guided)	16	2.9334
Otimizado (collapse + static)	16	3.8546
Otimizado (collapse + guided)	16	3.7746
Otimizado (collapse + dynamic)	16	45.1201

Tabela: O melhor resultado (16 threads) é apresentado para as versões paralelas.

Comparativo de Desempenho (16 Threads)

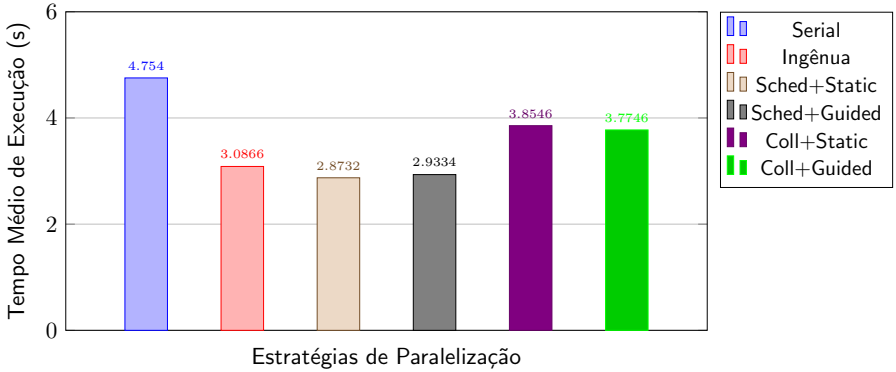


Figura: Comparativo do tempo de execução (16 threads). A estratégia `Sched+Static` apresentou o melhor desempenho.

Avaliação dos Resultados

Por que `static` foi rápido e `dynamic` foi lento?

A política de agendamento deve ser compatível com a natureza da carga de trabalho.

Cenário Ideal: Carga Uniforme

- **Problema:** Cada iteração do laço leva o mesmo tempo.
- **Solução:** `schedule(static)`.
- **Mecanismo:** O trabalho é dividido em blocos iguais **uma única vez**.
- **Resultado: Overhead mínimo**, melhor desempenho.

Cenário Inadequado: Carga Uniforme

- **Problema:** O mesmo, com carga uniforme.
- **Solução Incorreta:** `schedule(dynamic)`.
- **Mecanismo:** Cada thread pega **uma iteração de cada vez**.
- **Resultado: Overhead massivo**, desaceleração.

Conclusões

- **Estratégia > Micro-otimização:** A decisão de **quais laços** paralelizar foi o que gerou o maior ganho de desempenho.
- **O `schedule` é o fator mais crítico:** A comparação entre `static` (melhor desempenho) e `dynamic` (desaceleração acentuada) provou que a escolha do agendamento deve ser adequada à natureza da carga de trabalho.
- **Gargalos de Hardware:**A pequena diferença entre as versões otimizadas sugerem que o gargalo final não é o balanceamento de carga, mas sim a **largura de banda da memória** do sistema.

navier_stokes_serial.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h> // Usado no lugar de omp.h

#define NT 6000

// As funções auxiliares de alocação/liberação são as mesmas
double** allocate_grid(int nx, int ny) {
    double *data = (double*)malloc(nx * ny * sizeof(double));
    double **array = (double**)malloc(nx * sizeof(double*));
    for (int i = 0; i < nx; i++) {
        array[i] = &(data[i * ny]);
    }
    return array;
}

void free_grid(double** array) {
    free(array[0]);
    free(array);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Uso: %s <TAMANHO_DA_GRADE>\n", argv[0]);
        fprintf(stderr, "Exemplo: %s 512\n", argv[0]);
        return 1;
    }
    int NX = atoi(argv[1]);
    int NY = NX;

    // Aloca todas as 4 matrizes, para u e v
    double **u = allocate_grid(NX, NY);
    double **v = allocate_grid(NX, NY);
    double **u_new = allocate_grid(NX, NY);
    double **v_new = allocate_grid(NX, NY);

    // Inicialização idêntica à versão paralela, com u e v
    for (int i = 0; i < NX; i++) {
        for (int j = 0; j < NY; j++) {
            u[i][j] = 1.0; v[i][j] = 0.0;
            double dx = i - NX/2.0, dy = j - NY/2.0;
            double dist = sqrt(dx*dx + dy*dy);
            if (dist < (NX / 25.0)) {
                u[i][j] += 2.0 * exp(-dist*dist/(NX/5.0));
                v[i][j] += 1.5 * exp(-dist*dist/(NX/5.0));
            }
        }
    }

```

```c
    // Medição de tempo com clock()
    clock_t start_time = clock();

    for (int step = 0; step < NT; step++) {

        // --- CÁLCULO PRINCIPAL (SERIAL) ---
        for (int i = 1; i < NX-1; i++) {
            for (int j = 1; j < NY-1; j++) {
                double d2u_dx2 = (u[i+1][j] - 2.0*u[i][j] + u[i-1][j]);
                double d2u_dy2 = (u[i][j+1] - 2.0*u[i][j] + u[i][j-1]);
                double d2v_dx2 = (v[i+1][j] - 2.0*v[i][j] + v[i-1][j]);
                double d2v_dy2 = (v[i][j+1] - 2.0*v[i][j] + v[i][j-1]);

                // Usando o coeficiente de difusão ORIGINAL da versão paralela
                u_new[i][j] = u[i][j] + (0.001 * 0.01) * (d2u_dx2 + d2u_dy2);
                v_new[i][j] = v[i][j] + (0.001 * 0.01) * (d2v_dx2 + d2v_dy2);
            }
        }

        // --- CONDIÇÕES DE CONTORNO (SERIAL) ---
        // As duas seções agora rodam uma após a outra

        // Contorno Horizontal
        for (int i = 0; i < NX; i++) {
            u_new[i][0] = u_new[i][NY-2];
            u_new[i][NY-1] = u_new[i][1];
            v_new[i][0] = v_new[i][NY-2];
            v_new[i][NY-1] = v_new[i][1];
        }

        // Contorno Vertical
        for (int j = 0; j < NY; j++) {
            u_new[0][j] = u_new[NX-2][j];
            u_new[NX-1][j] = u_new[1][j];
            v_new[0][j] = v_new[NX-2][j];
            v_new[NX-1][j] = v_new[1][j];
        }

        // Troca de ponteiros para u e v
        double **temp_u = u;
        double **temp_v = v;
        u = u_new;
        v = v_new;
        u_new = temp_u;
        v_new = temp_v;
    }

    clock_t end_time = clock();
    double elapsed_time = (double)(end_time - start_time) / CLOCKS_PER_SEC;
    printf("%.6f\n", elapsed_time);

    // Libera a memória para todas as 4 matrizes
    free_grid(u); free_grid(v); free_grid(u_new); free_grid(v_new);
```

```c

    return 0;
}
```

navier_stokes_paralelo_ingenua.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define NT 6000

// As funções auxiliares de alocação/liberação são as mesmas
double** allocate_grid(int nx, int ny) {
    double *data = (double*)malloc(nx * ny * sizeof(double));
    double **array = (double**)malloc(nx * sizeof(double*));
    for (int i = 0; i < nx; i++) {
        array[i] = &(data[i * ny]);
    }
    return array;
}

void free_grid(double** array) {
    free(array[0]);
    free(array);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Uso: %s <TAMANHO_DA_GRADE>\n", argv[0]);
        fprintf(stderr, "Exemplo: %s 512\n", argv[0]);
        return 1;
    }
    int NX = atoi(argv[1]);
    int NY = NX;

    double **u = allocate_grid(NX, NY);
    double **v = allocate_grid(NX, NY);
    double **u_new = allocate_grid(NX, NY);
    double **v_new = allocate_grid(NX, NY);

    // Inicialização (pode ser paralela, não impacta muito a análise)
    #pragma omp parallel for
    for (int i = 0; i < NX; i++) {
        for (int j = 0; j < NY; j++) {
            u[i][j] = 1.0; v[i][j] = 0.0;
            double dx = i - NX/2.0, dy = j - NY/2.0;
            double dist = sqrt(dx*dx + dy*dy);
            if (dist < (NX / 25.0)) {
                u[i][j] += 2.0 * exp(-dist*dist/(NX/5.0));
                v[i][j] += 1.5 * exp(-dist*dist/(NX/5.0));
            }
        }
    }

```

```c
    double start_time = omp_get_wtime();

    for (int step = 0; step < NT; step++) {

        // --- INÍCIO DA ABORDAGEM INGÊNUA ---

        // GARGALO 1: Cria e destrói um time de threads apenas para este laço.
        #pragma omp parallel for
        for (int i = 1; i < NX-1; i++) {
            for (int j = 1; j < NY-1; j++) {
                double d2u_dx2 = (u[i+1][j] - 2.0*u[i][j] + u[i-1][j]);
                double d2u_dy2 = (u[i][j+1] - 2.0*u[i][j] + u[i][j-1]);
                double d2v_dx2 = (v[i+1][j] - 2.0*v[i][j] + v[i-1][j]);
                double d2v_dy2 = (v[i][j+1] - 2.0*v[i][j] + v[i][j-1]);

                u_new[i][j] = u[i][j] + (0.001 * 0.01) * (d2u_dx2 + d2u_dy2);
                v_new[i][j] = v[i][j] + (0.001 * 0.01) * (d2v_dx2 + d2v_dy2);
            }
        }

        // GARGALO 2: Cria e destrói OUTRO time de threads apenas para as seções.
        #pragma omp parallel sections
        {
            #pragma omp section
            { // Contorno Horizontal
                for (int i = 0; i < NX; i++) {
                    u_new[i][0] = u_new[i][NY-2];
                    u_new[i][NY-1] = u_new[i][1];
                    v_new[i][0] = v_new[i][NY-2];
                    v_new[i][NY-1] = v_new[i][1];
                }
            }
            #pragma omp section
            { // Contorno Vertical
                for (int j = 0; j < NY; j++) {
                    u_new[0][j] = u_new[NX-2][j];
                    u_new[NX-1][j] = u_new[1][j];
                    v_new[0][j] = v_new[NX-2][j];
                    v_new[NX-1][j] = v_new[1][j];
                }
            }
        }

        // A troca de ponteiros ocorre serialmente, pela thread mestre, após os joins.
        double **temp_u = u;
        double **temp_v = v;
        u = u_new;
        v = v_new;
        u_new = temp_u;
        v_new = temp_v;

```

```
        // --- FIM DA ABORDAGEM INGÊNUA ---
    }

    double end_time = omp_get_wtime();
    printf("%.6f\n", end_time - start_time);

    free_grid(u); free_grid(v); free_grid(u_new); free_grid(v_new);

    return 0;
}

```

navier_stokes_S_static.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

// NT agora é uma constante interna.
#define NT 6000

// Funções auxiliares agora recebem as dimensões como parâmetros
double** allocate_grid(int nx, int ny) {
    double *data = (double*)malloc(nx * ny * sizeof(double));
    double **array = (double**)malloc(nx * sizeof(double*));
    for (int i = 0; i < nx; i++) {
        array[i] = &(data[i * ny]);
    }
    return array;
}

void free_grid(double** array) {
    free(array[0]);
    free(array);
}

int main(int argc, char *argv[]) {
    // Agora esperamos o tamanho da grade (NX) como argumento
    if (argc != 2) {
        fprintf(stderr, "Uso: %s <TAMANHO_DA_GRADE>\n", argv[0]);
        fprintf(stderr, "Exemplo: %s 512\n", argv[0]);
        return 1;
    }
    int NX = atoi(argv[1]);
    int NY = NX; // NY será sempre igual a NX

    // Alocação de memória usa as variáveis NX e NY
    double **u = allocate_grid(NX, NY);
    double **v = allocate_grid(NX, NY);
    double **u_new = allocate_grid(NX, NY);
    double **v_new = allocate_grid(NX, NY);

    // Inicialização usa as variáveis NX e NY
    #pragma omp parallel for
    for (int i = 0; i < NX; i++) {
        for (int j = 0; j < NY; j++) {
            u[i][j] = 1.0; v[i][j] = 0.0;
            double dx = i - NX/2.0, dy = j - NY/2.0;
            double dist = sqrt(dx*dx + dy*dy);
            if (dist < (NX / 25.0)) { // Condição inicial relativa ao tamanho da grade
                u[i][j] += 2.0 * exp(-dist*dist/(NX/5.0));
                v[i][j] += 1.5 * exp(-dist*dist/(NX/5.0));
```

```
            }
        }
    }

    double start_time = omp_get_wtime();

    #pragma omp parallel
    {
        for (int step = 0; step < NT; step++) {

            #pragma omp for schedule(static)
            for (int i = 1; i < NX-1; i++) {
                for (int j = 1; j < NY-1; j++) {
                    double d2u_dx2 = (u[i+1][j] - 2.0*u[i][j] + u[i-1][j]);
                    double d2u_dy2 = (u[i][j+1] - 2.0*u[i][j] + u[i][j-1]);
                    double d2v_dx2 = (v[i+1][j] - 2.0*v[i][j] + v[i-1][j]);
                    double d2v_dy2 = (v[i][j+1] - 2.0*v[i][j] + v[i-1][j]);

                    u_new[i][j] = u[i][j] + (0.001 * 0.01) * (d2u_dx2 + d2u_dy2); // DT e NU podem ser fixados
                    v_new[i][j] = v[i][j] + (0.001 * 0.01) * (d2v_dx2 + d2v_dy2);
                }
            }

            #pragma omp for schedule(static)
            for (int i = 0; i < NX; i++) {
                u_new[i][0] = u_new[i][NY-2];
                u_new[i][NY-1] = u_new[i][1];
                v_new[i][0] = v_new[i][NY-2];
                v_new[i][NY-1] = v_new[i][1];
            }

            #pragma omp for schedule(static)
            for (int j = 0; j < NY; j++) {
                u_new[0][j] = u_new[NX-2][j];
                u_new[NX-1][j] = u_new[1][j];
                v_new[0][j] = v_new[NX-2][j];
                v_new[NX-1][j] = v_new[1][j];
            }

            #pragma omp single
            {
                double **temp_u = u;
                double **temp_v = v;
                u = u_new;
                v = v_new;
                u_new = temp_u;
                v_new = temp_v;
            }
        }
    }

```

```
    double end_time = omp_get_wtime();
    printf("%.6f\n", end_time - start_time);

    free_grid(u); free_grid(v); free_grid(u_new); free_grid(v_new);

    return 0;
}

```

navier_stokes_C_S_static.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

// NT agora é uma constante interna.
#define NT 6000

// Funções auxiliares agora recebem as dimensões como parâmetros
double** allocate_grid(int nx, int ny) {
    double *data = (double*)malloc(nx * ny * sizeof(double));
    double **array = (double**)malloc(nx * sizeof(double*));
    for (int i = 0; i < nx; i++) {
        array[i] = &(data[i * ny]);
    }
    return array;
}

void free_grid(double** array) {
    free(array[0]);
    free(array);
}

int main(int argc, char *argv[]) {
    // Agora esperamos o tamanho da grade (NX) como argumento
    if (argc != 2) {
        fprintf(stderr, "Uso: %s <TAMANHO_DA_GRADE>\n", argv[0]);
        fprintf(stderr, "Exemplo: %s 512\n", argv[0]);
        return 1;
    }
    int NX = atoi(argv[1]);
    int NY = NX; // NY será sempre igual a NX

    // Alocação de memória usa as variáveis NX e NY
    double **u = allocate_grid(NX, NY);
    double **v = allocate_grid(NX, NY);
    double **u_new = allocate_grid(NX, NY);
    double **v_new = allocate_grid(NX, NY);

    // Inicialização usa as variáveis NX e NY
    #pragma omp parallel for
    for (int i = 0; i < NX; i++) {
        for (int j = 0; j < NY; j++) {
            u[i][j] = 1.0; v[i][j] = 0.0;
            double dx = i - NX/2.0, dy = j - NY/2.0;
            double dist = sqrt(dx*dx + dy*dy);
            if (dist < (NX / 25.0)) { // Condição inicial relativa ao tamanho da grade
                u[i][j] += 2.0 * exp(-dist*dist/(NX/5.0));
                v[i][j] += 1.5 * exp(-dist*dist/(NX/5.0));
```

```
            }
        }
    }

    double start_time = omp_get_wtime();

    #pragma omp parallel
    {
        for (int step = 0; step < NT; step++) {

            #pragma omp for collapse(2) schedule(static)
            for (int i = 1; i < NX-1; i++) {
                for (int j = 1; j < NY-1; j++) {
                    double d2u_dx2 = (u[i+1][j] - 2.0*u[i][j] + u[i-1][j]);
                    double d2u_dy2 = (u[i][j+1] - 2.0*u[i][j] + u[i][j-1]);
                    double d2v_dx2 = (v[i+1][j] - 2.0*v[i][j] + v[i-1][j]);
                    double d2v_dy2 = (v[i][j+1] - 2.0*v[i][j] + v[i-1][j]);

                    u_new[i][j] = u[i][j] + (0.001 * 0.01) * (d2u_dx2 + d2u_dy2); // DT e NU podem ser fixados
                    v_new[i][j] = v[i][j] + (0.001 * 0.01) * (d2v_dx2 + d2v_dy2);
                }
            }

            #pragma omp for schedule(static)
            for (int i = 0; i < NX; i++) {
                u_new[i][0] = u_new[i][NY-2];
                u_new[i][NY-1] = u_new[i][1];
                v_new[i][0] = v_new[i][NY-2];
                v_new[i][NY-1] = v_new[i][1];
            }

            #pragma omp for schedule(static)
            for (int j = 0; j < NY; j++) {
                u_new[0][j] = u_new[NX-2][j];
                u_new[NX-1][j] = u_new[1][j];
                v_new[0][j] = v_new[NX-2][j];
                v_new[NX-1][j] = v_new[1][j];
            }

            #pragma omp single
            {
                double **temp_u = u;
                double **temp_v = v;
                u = u_new;
                v = v_new;
                u_new = temp_u;
                v_new = temp_v;
            }
        }
    }

```

```
    double end_time = omp_get_wtime();
    printf("%.6f\n", end_time - start_time);

    free_grid(u); free_grid(v); free_grid(u_new); free_grid(v_new);

    return 0;
}

```

calcular_media_paralelo.sh

```bash
#!/bin/bash

# --- PARÂMETROS DE CONFIGURAÇÃO ---
# Nome do arquivo de código-fonte C que será compilado.
CODIGO_C="navier_stokes_paralelo_ingenua.c"

# Nome que daremos ao programa executável após a compilação.
EXECUTAVEL="./navier_stokes_paralelo_ingenua"

# Número de vezes que cada configuração será executada para calcular a média.
NUM_REPETICOES=5

# Lista de números de threads para testar.
LISTA_THREADS="16"

# Parâmetro do tamanho da grade a ser passado para o executável.
#
TAMANHO_GRADE=512


# --- 1. COMPILAÇÃO DO CÓDIGO ---
echo "Compilando o código '$CODIGO_C' com otimização (-O3) e OpenMP..."
if ! gcc -o $EXECUTAVEL $CODIGO_C -lm -fopenmp -O3; then
    echo "Erro de compilação. Abortando."
    exit 1
fi
echo "Compilação bem-sucedida."
echo


# --- 2. EXECUÇÃO E CÁLCULO DA MÉDIA ---
for threads in $LISTA_THREADS
do
    echo "-------------------------------------"
    echo "Iniciando testes com $threads thread(s) e grade $TAMANHO_GRADE..."

    TOTAL_TIME=0.0

    for i in $(seq 1 $NUM_REPETICOES)
    do
        # Executa o programa passando o TAMANHO_GRADE como argumento
        #
        TEMPO_EXECUCAO=$(OMP_NUM_THREADS=$threads $EXECUTAVEL $TAMANHO_GRADE)

        echo "  -> Repetição $i: $TEMPO_EXECUCAO s"

        TOTAL_TIME=$(echo "$TOTAL_TIME + $TEMPO_EXECUCAO" | bc)
    done

    MEDIA=$(echo "scale=6; $TOTAL_TIME / $NUM_REPETICOES" | bc)
```

```
    echo ""
    echo "Tempo Médio para $threads thread(s): $MEDIA s"
done


# --- 3. LIMPEZA ---
echo "-------------------------------------"
echo "Bateria de testes concluída."
rm $EXECUTAVEL
echo "Executável temporário '$EXECUTAVEL' removido."
```